

CONFIDENTIAL — INTERNAL ENGINEERING REPORT

Carland Service

Code Health & Architecture Audit

Full-codebase review: structure, duplication, dead code, security posture and maintainability

Project	carland_service (Spring Boot 3.4.3, Java 21)
Repository	github.com/azizJava91/carcat-service
Audit date	July 31, 2026
Method	Static review of every package, class and method in <code>src/main/java</code> , resources and build/deploy configuration. No code was modified.
Audience	Engineering management

1 Executive Summary

carland_service is the core backend of the Carland/CarCat product: it manages cars, customers, maintenance templates, service-life percentages, partner service visits (Hyper integration), photos, notifications and an admin panel. The service is **functional and actively deployed**, and several recent additions (webhook HMAC security, the V2 visit model) show good engineering instincts. However, the codebase carries significant accumulated debt that now slows every change and creates real production risk.

~230

JAVA FILES (MAIN APP)

~77

ACTIVE API ENDPOINTS

2,038

LINES IN LARGEST CLASS

42%

OF REPO IS A STALE COPY

1

TEST (EMPTY, SKIPPED IN CI)

0

BEAN-VALIDATION CHECKS

Top 5 risks (ranked)

SEVERITY	FINDING	WHY IT MATTERS
CRITICAL	API authentication does not verify tokens — any string starting with <code>Bearer</code> is accepted	Every "protected" endpoint (cars, records, photos, personal data) is effectively open to anyone who sends a fake header. Identity is taken from client-supplied headers.
CRITICAL	Admin panel guarded only by hardcoded credentials (<code>nemet</code> / <code>nemet</code>) committed to source	Anyone with repo access or who guesses the credentials gets the full customer/car database and Excel exports. <code>/admin/**</code> is also excluded from Spring Security entirely.
CRITICAL	A full stale copy of the repository (<code>carcat-service/</code> , 205 tracked files) is committed inside the repository	42% of tracked files are dead weight. It corrupts code search, invites edits to the wrong file, and doubles confusion for every new developer.
HIGH	<code>CarServiceImpl</code> is a 2,038-line god class; its two percentage methods alone are ~660 lines	The core business calculation (service-life percentages) is implemented twice with different formulas, so results can disagree depending on which endpoint is called.
HIGH	No safety net: 1 empty test, CI builds with <code>-x test</code> , schema managed by <code>ddl-auto: update</code>	Every deploy is unverified; the database schema mutates automatically in production with no migration history or rollback path.

Overall assessment

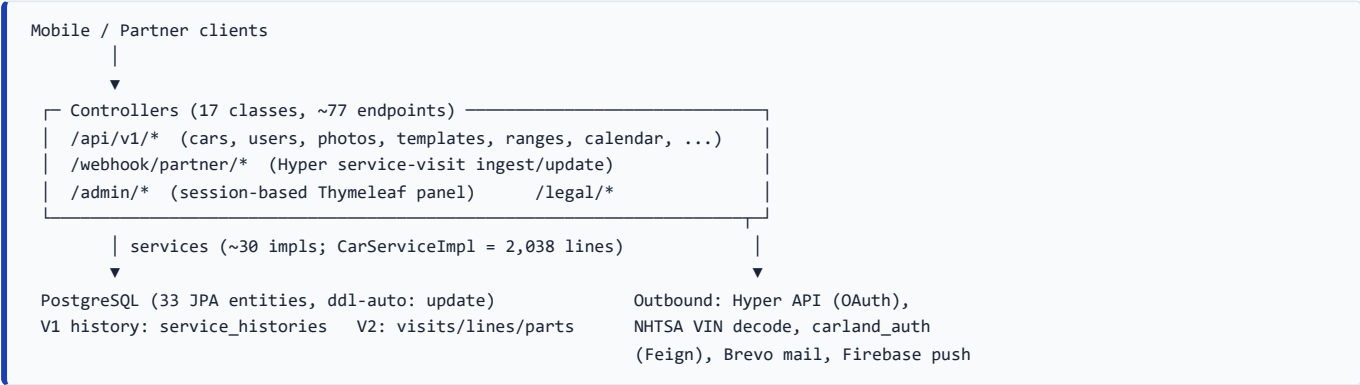
The product logic is real and valuable, but the codebase is in a state where **correctness depends on individual memory rather than structure**: business rules are duplicated in 2–3 places, security is inconsistent per endpoint, roughly 10% of classes are dead or stubbed, and there is no automated verification. None of this requires a rewrite — the recommended path (Section 10) is a focused 4-phase cleanup that can be executed incrementally alongside feature work.

2 Scope & Method

- Reviewed:** all packages under `src/main/java/com/carland/carland_service` (controller, service, entity, repository, dto, enums, exceptions, filter, config, jwt, util, feign), `src/main/resources` , `build.gradle` , Dockerfile and the GitHub Actions pipeline.
- Excluded from deep review:** the nested `carcat-service/` folder (confirmed to be a stale duplicate — reported as a finding, not analyzed line-by-line) and the separate `carland_auth` project (referenced only where the two services interact).
- Method:** full manual read of the largest classes, systematic usage searches for every exception, repository method and DTO, cross-file duplication comparison, and configuration/security review. **No code was changed.**

3 Architecture Overview

The service follows a conventional Spring Boot layering on paper — controller → service interface → service implementation → repository → entity — with four outbound integrations and one inbound webhook channel:



Security is split into three inconsistent regimes:

SURFACE	PROTECTION	ASSESSMENT
/api/v1/**	Custom filter checks only that an <code>Authorization: Bearer ...</code> header exists; the JWT is never parsed or verified	CRITICAL
/admin/**	Excluded from Spring Security; HTTP-session flag set after comparing to hardcoded credentials	CRITICAL
/webhook/partner/*	Two-layer: shared <code>X-Internal-Token</code> + per-partner HMAC-SHA256 signature (constant-time compares, secrets in DB/env)	GOOD

The webhook security stack (internal token + HMAC, fail-closed validation, structured 401 responses) is the best-engineered part of the codebase and should be the model for the rest.

4 Critical Findings (act immediately)

4.1 Token "validation" that validates nothing

jwt/CustomFilter.java — any Bearer string authenticates

The filter checks `authHeader.startsWith("Bearer ")` and then unconditionally creates an authenticated principal named `"user"`. The three `jjwt` libraries declared in `build.gradle` are never used. Real user identity (user id, phone, role) is taken from *client-supplied request headers* (`X-User-Id`, `phoneNumber`, `role`), which any caller can forge. In practice, all "authenticated" endpoints are open, and any user can act as any other user by changing a header.

4.2 Admin panel: hardcoded credentials + security bypass

AdminController.java line 67 + JWTConfiguration permitAll

`if ("nemet".equals(username) && "nemet".equals(password))` is the entire admin login. The credentials are committed to a repository whose history is shared; there is no hashing, rate-limiting or audit trail. `/admin/**` is additionally listed as `permitAll` in Spring Security and bypassed in the JWT filter, so the panel's only protection is this string comparison. Behind it sit the full car and customer databases plus one-click Excel exports.

4.3 Stale nested repository copy committed to git

carcat-service/ — 205 tracked files, ~42% of the repository

A complete, older fork of the project (including its own `.github/workflows`, Dockerfile and full source tree) lives inside the repository and is fully tracked by git. File contents have diverged from the live tree (verified by hash comparison). Consequences observed during this audit: code searches return double results, and it is easy to open and edit the dead copy by mistake. It should be deleted and git-ignored.

4.4 Sensitive data written to logs

LOCATION	WHAT LEAKS
<code>UserController</code> (~L103–111)	Logs the full Authorization token and all request headers
<code>util/MailTrapConfig</code>	Prints the mail API token to <code>System.err</code> at startup
<code>jwt/CustomFilter</code>	Prints every request URI via <code>System.out.println</code> (noise, and URIs can contain identifiers)
<code>application.yaml</code>	<code>show_sql: true</code> dumps all SQL (with data patterns) to production logs

4.5 Unprotected bulk data exports

- `GET /api/v1/user/customer-cars` — generates an Excel file of **all customers and their cars** and is on the filter's *public* path list (no authentication at all).
- `GET /api/v1/user/get/all/with/cars` — full customer+car dump behind the non-validating Bearer check only.
- Admin Excel exports (`/admin/cars/export`, `/admin/users/export`) sit behind the hardcoded password only.

5 Structural Debt: God Classes & Spaghetti Logic

5.1 CarServiceImpl — 2,038 lines

The single most expensive maintenance point in the system. It owns 17 public operations plus helpers, mixing authentication, validation, business math, persistence, response mapping and even push-notification copywriting.

METHOD	APPROX. SIZE	MIXED RESPONSIBILITIES
<code>executeServicePercentages</code> (L827–1171)	~345 lines	Auth, status branching, partner sync, record/history merging, percentage math, persistence, response building, a manual re-read to dodge a race condition
<code>getServicePercentageList</code> (L413–730)	~318 lines	Re-implements the same km/month math <i>inline</i> instead of reusing the existing helper methods
<code>addCar</code> (L1176–1454)	~279 lines	Validation, customer/color/plate/VIN checks, template bootstrap, percentage + record creation, DB audit logging, async post-commit sync
<code>checkVin</code> (L66–174)	~109 lines	DB check + NHTSA decode + field extraction + response assembly

Business-correctness risk, not just style

The percentage formula exists in **three places**: inline in `getServicePercentageList`, in the `computeKmPercentage` / `computeMonthPercentage` helpers used by `executeServicePercentages`, and again in `HyperPercentageSyncService.applyPartnerData`. The list and execute paths already use *different month formulas* in their default branches, so the value a user sees in the list can differ from the value stored after execution.

5.2 Other oversized / tangled classes

CLASS	LINES	PROBLEM
<code>controller/Legal.java</code>	558	~475 lines of terms & privacy-policy text embedded as Java text blocks; legal edits require recompile + redeploy; also contains a debug endpoint returning <code>"AAHAHAHAHA..."</code>
<code>PhotoServiceImpl</code>	459	Four near-identical upload/get/delete pipelines (car / user / partner / badge-logo); media-type normalization copy-pasted 4×; returns <code>ResponseEntity</code> from the service layer
<code>dto/response/v2/CarVinHistoryServiceV2Impl</code>	414	A full <code>@Service</code> living inside a DTO package (see 5.3)
<code>PartnerServiceVisitUpdateServiceImpl</code>	369	~55-line procedural entry method; ~55 lines of repetitive field-diff blocks; ownership failure disguised as 404
<code>AdminController</code>	353	MVC controller doing repository pagination, in-memory paging of Feign results and full Apache POI Excel generation inline
<code>HyperPercentageSyncService</code>	341	Sync + matching + a third copy of the percentage math

5.3 Misplaced classes (package hygiene)

CLASS	ACTUAL KIND	LIVES IN	BELONGS IN
<code>Visit</code> , <code>ServiceHistoryV2</code> , <code>ServiceHistoryPartV2</code>	JPA entities (<code>visits</code> , <code>visit_service_lines</code> , <code>visit_parts</code>)	<code>dto.responses.v2</code>	<code>entity</code>
<code>CarVinHistoryServiceV2</code> / <code>...V2Impl</code>	Service interface + implementation	<code>dto.responses.v2</code>	<code>service</code> / <code>service.impl</code>
<code>MailTrapConfig</code>	<code>@Configuration</code> (also defines the app's only <code>RestTemplate</code> bean)	<code>util</code>	<code>config</code>

PushCron	Scheduled job (fully commented out)	util	job / delete
Helper	@Service doing user provisioning by role + date formatting	util	split into proper services
FirebaseTokenService	@Configuration named like a service	service.impl 1	config

6 Duplicated Logic Inventory

DUPLICATION	COPIES	EVIDENCE / IMPACT
Service-life percentage math (km + month formulas)	3	Inline in <code>getServicePercentageList</code> ; helpers in <code>CarServiceImpl</code> L1889–1916; again in <code>HyperPercentageSyncService</code> L286–311. Formulas already diverge.
Hyper VIN-history flow (fetch, lock, persist, map)	2	<code>CarVinHistoryServiceImpl</code> (276 lines, V1) vs <code>CarVinHistoryServiceV2Impl</code> (414 lines, V2): identical <code>fetchHyperHistory</code> HTTP method byte-for-byte, parallel persist/map pipelines. Hyper→DTO mapping additionally exists a third time in <code>HyperWebhookIngestMapper</code> .
Customer authentication boilerplate (<code>findByIdAndPhoneNumberAndStatus</code> + null check)	~14	Repeated at the top of nearly every <code>CarServiceImpl</code> method instead of one guard/helper.
Webhook ingest vs update services	2	~25 lines of byte-identical mapper helpers (<code>mapLineToEntity</code> , <code>mapPartToEntity</code> , <code>normalizeUniversalServiceId</code> , cost recalculation) plus a ~15-line identical entry pipeline across 604 combined lines.
Response-builder blocks	6+ / 4	<code>CarServicePercentageResponse.builder()</code> ... appears ≥6 times; <code>RecordResponse.builder()</code> ... 4 times — one mapper each would do.
Photo upload/get pipeline & media-type normalization	4	Copy-pasted per photo kind in <code>PhotoServiceImpl</code> .
Body-buffering filters	2	<code>ContentCachingFilter</code> wraps every request (incl. 50 MB multipart) at highest precedence; <code>WebhookSignatureFilter</code> then buffers the same body again via <code>CachedBodyHttpServletRequest</code> — double memory per webhook.
Person-shaped DTOs	3	<code>NameSurname</code> , <code>AuthUser</code> , <code>CustomerResponse</code> model the same person with overlapping fields.
V1 vs V2 history response DTOs	2 sets	<code>CarVinServiceHistoryResponse</code> / <code>ServiceHistoryItemResponse</code> / <code>...PartResponse</code> vs their V2 twins — both API versions live under <code>/api/v1</code> .

7 Dead, Stubbed & Unused Code

7.1 Endpoints that return null or nonsense

SURFACE	STATE
<code>SuperAdminController</code> + service + 2 DTOs	Controller has zero endpoints ; service returns <code>null</code> ; both DTOs have zero fields — an entire dead vertical slice
<code>AppointmentServiceImpl</code> (both methods), <code>AutoServiceServiceImpl.getService</code> / <code>addServiceAmount</code> , <code>RangeServiceImpl.decideOnBooking</code> , <code>MaintenanceTemplateServiceImpl.createMaintenanceTemplate</code>	All <code>return null;</code> — live endpoints backed by stubs (real bodies sit in comments)
<code>GET /legal/get</code> , <code>GET /webhook/partner/test</code> , <code>GET /api/v1/car/test/hyper</code>	Debug/test endpoints in production route table; <code>/test/hyper</code> calls the real Hyper API with a hardcoded VIN and is on the public path list

7.2 Unused classes and members (usage-verified)

- **9 exception classes never thrown anywhere** — `ExpiredOtpException` , `InvalidOtpCodeException` , `RefreshTokenNotSetException` , `UsernameAlreadyExistException` , `MsmTransactionException` , `InviteException` , `WrongPasswordException` , `UnderageException` , `NotificationException` . All are OTP/invite/auth concerns evidently copied from `carland_auth`; each still has a handler registered in the 256-line `CustomExceptionHandler` . `UICustomExceptionHandler` renders a template (`invite-error`) that does not exist.

- **Unused repository methods:** `BrandRepository.findAllByBrandName / existsByBrandName`, `PartnerRepository.findBySourceIgnoreCaseAndActiveTrue`, `ColorRepository.findAllByOrderByColorIdAsc`, `VisitRepository.findWithDetailsByCarAndHyperRecordId`, `ModelYearRepository.findAllByStatus`.
- **Dead config/util:** `RestTemplateConfig` (an empty class — no annotation, no beans), `CustomImageCrop` (only call sites are commented), `PushCron` (all schedules commented), the Mailtrap client bean (never injected), `ServiceNameAz` enum (entire body commented out).
- **Dead notification pipeline:** `CarServiceImpl.canSendNotification` and `buildMessage` (~85 lines) are referenced only from commented code, while the *live* `calculateAndPushNotification` sends a hardcoded fake message ("oil change — 230 km left") to every car with a device token. Only the fact that its cron trigger is also commented out prevents mass fake pushes.
- **Unused DTOs:** `PrivacyPolicyResponse`, `TermsConditionsResponse` (imported but never used), SuperAdmin request/response.
- **Unused controller members:** `CarController` injects `customerRepository` and `maintenanceTemplateRepository` that are never read; ~30 `@RequestHeader("Authorization")` parameters across 7 controllers are accepted and then ignored.
- **Unused build dependencies:** `jjwt-api/impl/jackson` (no JWT parsing exists), `spring-boot-starter-webflux` (no WebClient/Mono/Flux usage), `mailtrap-java` and `spring-boot-starter-mail` (active mail path is Brevo via RestTemplate), Lombok declared twice.

7.3 Commented-out code volume

12 blocks longer than 3 lines, the largest being ~80 and ~68 lines of legacy doctor/appointment logic in `RangeServiceImpl`, ~44 lines in `CarServiceImpl`, ~35 in `MaintenanceTemplateServiceImpl`, plus fully commented files (`ServiceNameAz`, `PushCron` schedules, alternate HMAC strategies). Commented code is where the real business rules currently hide — e.g. template-creation auth checks are commented out, meaning **role verification for maintenance-template management is disabled in production**.

8 Data Layer & API Design

8.1 Persistence findings

FINDING	DETAIL	SEVERITY
No migration tool	Liquibase is commented out in <code>build.gradle</code> ; production schema evolves via <code>ddl-auto: update</code> — no history, no rollback, silent drift	HIGH
Lombok <code>@Data</code> on all 30 entities	Generated equals/hashCode/toString over JPA associations risk LazyInitialization errors, recursion and broken collection semantics; only sporadic <code>@ToString.Exclude</code> mitigations	MEDIUM
Raw <code>Long</code> foreign keys	<code>Car.colorId / engineTypeId</code> , <code>Percentage.carId / serviceId</code> , <code>Notification.customerId</code> , photo tables etc. bypass JPA relations while proper <code>@ManyToOne</code> is used elsewhere — two inconsistent styles	MEDIUM
Denormalized car attributes	<code>Car.brand/model/engineType/bodyType/transmissionType</code> stored as free strings although lookup tables exist for each	MEDIUM
Zero declared indexes	No <code>@Index</code> anywhere; frequent lookups (percentages by car, notifications by customer) rely on defaults	MEDIUM
<code>Optional</code> inconsistency	<code>CarRepository</code> mixes <code>Optional<Car> findByPlateNumberIgnoreCase</code> with null-returning <code>findByVin</code> ; callers null-check inconsistently	LOW
Transactions	Only <code>addCar</code> is <code>@Transactional</code> in <code>CarServiceImpl</code> ; multi-write loops (<code>executeServicePercentages</code>) can partially commit. Conversely, VIN-history and feedback services hold DB transactions <i>across external HTTP calls</i> . One service uses <code>jakarta.transaction.Transactional</code> instead of Spring's	HIGH
N+1 query patterns	Per-item queries in percentage list/execute, per-row part loading in V1 history mapping, per-car color lookup in car-list conversion, per-car device-token lookup in the notification loop	MEDIUM

8.2 API design findings

- **RPC-style verb URLs** throughout: `/get/brand/list` , `/for/car/upload` , `/insert/service/history` , `/edit/byId` — no resource-oriented convention.
- **Wrong HTTP methods:** reads via `PUT` (`PUT /api/v1/car/get/record`), deletes via `PUT` (`/car/remove` , `/range/delete`), updates via `POST` , and `GET` endpoints that require request bodies (often stripped by proxies).
- **Zero bean validation:** not a single `@Valid` / `jakarta.validation` annotation in the controller or DTO trees; all validation is manual and inconsistent.
- **Entities leaked as API responses:** `GroupByController` , `NotificationController` and `getColors` return JPA entities directly, coupling the public contract to the DB schema.
- **Version confusion:** V2 history exposed as a path suffix under V1 (`/api/v1/car/{vin}/service-history/v2`); `/webhook` , `/admin` , `/legal` unversioned.
- **Three different error envelopes** face integrators: webhook auth errors, global `ResponseException` payloads and success-shaped bodies returned with 409 (which is also used for benign no-op updates).

8.3 Hardcoded business data in enums

`EnumMessagesLangValues` holds 90 message constants in three languages — including leftover *hospital/doctor-domain* messages (`HOSPITAL_NOT_FOUND` , `ONLY_DOCTOR_ALLOWED` ...) from a previous product. Five more enums hardcode translations and partner data (`ServiceTypeTranslation` 37 entries, `ColorTranslation` 19, `HyperServiceMapping` 23 alias mappings — one of which is ambiguous and silently returns empty on collision, `EnumPartnerId` duplicating the `partners` DB table). Adding a partner or fixing a translation currently requires a code deploy.

9 Engineering Practices

PRACTICE	CURRENT STATE	RATING
Automated tests	One file with an empty <code>contextLoads()</code> ; CI pipeline builds with <code>-x test</code> , so even that never runs	CRITICAL

CI/CD	GitHub Actions → Docker Hub → SSH docker-compose deploy on every push to <code>main</code> : solid automation, but no quality gate of any kind before production	MEDIUM
Logging	Mix of SLF4J, 5× <code>System.out.println</code> , 15× <code>System.err.println</code> ; debug banners (<code>"000000..."</code> , <code>"wwwwww..."</code>); log messages in three languages incl. informal strings ("yuxudan oyandim rabbit istek gonderdi")	MEDIUM
Error handling	Generic <code>RuntimeException</code> in mail/push services; <code>HyperTokenService</code> swallows failures and returns <code>null</code> tokens; duplicate webhook <code>serviceCode</code> 's are logged and ingested anyway	HIGH
Naming & language	Identifiers in English but user-facing strings, log lines and even exception messages mix Azerbaijani, Turkish and English; typos (<code>FeedBackResponse</code>); class named <code>Legal</code> instead of <code>LegalController</code> ; misleading names (<code>getCalendarByDoctorId</code> serving auto-services)	LOW
Caching / scale readiness	Hyper OAuth token cached in a local in-memory map — with Hyper's 20 req/hour limit, running a second instance would cause token thrash	MEDIUM

10 Recommended Remediation Plan

The plan is ordered so that each phase is independently shippable and reduces risk before convenience. Estimates assume one experienced developer familiar with the codebase.

Phase 1 — Security & hygiene (≈ 1 week)

1. Implement real JWT validation in `CustomFilter` (the `jjwt` dependencies are already on the classpath), or validate against `carland_auth`; stop trusting client-supplied identity headers.
2. Move admin credentials to environment variables with a hashed password; add rate limiting; remove `/admin/**` from the public path lists.
3. Remove the token/header logging in `UserController`, the mail-token print in `MailTrapConfig`, and the per-request `System.out` in the filter.
4. Lock down or delete the public bulk exports (`/customer-cars`) and the test/debug endpoints (`/test/hyper`, `/legal/get`).
5. Delete `carcat-service/` from the repository and add it to `.gitignore`.

Phase 2 — Safety net (≈ 1–2 weeks)

1. Re-enable tests in CI (drop `-x test`); add characterization tests around the percentage math and webhook ingest/update before touching them.
2. Introduce Liquibase (already half-planned — the dependency is commented out); snapshot the current schema as the baseline; switch `ddl-au` to `validate`.
3. Turn off `show_sql`; standardize on SLF4J.

Phase 3 — Dead-code purge (≈ 3–4 days, low risk)

1. Delete: 9 unused exceptions + their handlers, unused repository methods, empty `RestTemplateConfig`, `PushCron`, `CustomImageCrop`, `ServiceNameAz`, the SuperAdmin slice, unused DTOs, unused controller fields and ~30 ignored Authorization parameters.
2. Remove unused dependencies (`webflux`, `jjwt` if unused after Phase 1, `mailtrap`, `spring-mail`, duplicate Lombok line).
3. Delete or implement the `return null` stub endpoints; either restore or remove the commented notification pipeline and the commented template-auth checks (decide, don't defer).

Phase 4 — Structural refactoring (incremental, ≈ 3–4 weeks spread out)

1. Extract from `CarServiceImpl`: a `CustomerAccessGuard` (kills ~14 duplications), a single `PercentageCalculator` (kills the triple formula), and response mappers.
2. Move the V2 entities and service out of `dto.response.v2` into `entity / service`; decide the V1-history sunset plan and collapse the duplicated Hyper fetch/map code into one client + one mapper.
3. Extract shared webhook persistence support from the ingest/update twins; move Excel generation out of `AdminController` into an export service.
4. Externalize legal texts (static resources or DB), translations and partner/service mappings out of Java enums.
5. Adopt `@Valid` bean validation on request DTOs; stop returning JPA entities from controllers.

What NOT to do

A big-bang rewrite is not justified. The domain logic works and is encoded in this code (including in its commented-out corners). The debt is concentrated: two security fixes, one folder deletion, one god class and one duplicated integration account for most of the risk. Fix those and the codebase becomes ordinary.

11 Appendix — Key Metrics

METRIC	VALUE
Java files in live source tree (<code>src/main/java</code>)	~230

Git-tracked files in stale <code>carcat-service/</code> copy	205 (≈ 42% of all tracked files)
Controllers / active endpoints	17 / ~77 (Car 19, Photo 11, Admin 7, GroupBy 7, Webhook 4, others smaller; SuperAdmin 0)
Largest classes	<code>CarServiceImpl</code> 2,038 · <code>Legal</code> 558 · <code>PhotoServiceImpl</code> 459 · <code>CarVinHistoryServiceV2Impl</code> 414 · <code>PartnerServiceVisitUpdateServiceImpl</code> 369 · <code>AdminController</code> 353
Methods over 100 lines in <code>CarServiceImpl</code>	5 (two exceed 300 lines; ~32% of the class is two methods)
JPA entities / repositories	33 (3 misplaced in DTO package) / 32 (all referenced; 6 unused query methods)
Exception classes / never thrown	23 / 9
Message/translation constants hardcoded in enums	90 messages × 3 languages + 5 translation/mapping enums (incl. leftover hospital-domain entries)
<code>System.out</code> / <code>System.err</code> / <code>printStackTrace</code>	5 / 15 / 0
Commented-out blocks > 3 lines	12 (largest ≈ 80 lines)
Bean-validation (<code>@Valid</code>) usages	0
Test files / tests executed in CI	1 (empty) / 0 (CI uses <code>-x test</code>)
Unused build dependencies	4 (webflux, jjwt×3 counted as one, mailtrap, spring-mail) + duplicate Lombok declaration

Prepared by the engineering team's automated audit tooling with full manual verification of all cited locations. All file paths and line numbers refer to the repository state on July 31, 2026. No source files were modified during this audit.